

DragonSR: Directed Acyclic Graph Search for Equation Discovery

Elyas Chikhaoui, Julie Keisler, Anastase Charantonis

2026

Abstract

Discovering underlying equations from observational data is a fundamental challenge in experimental physics, that can lead to deeper understanding, improved predictive power, and better generalization of physical phenomena. Symbolic regression (SR) addresses this challenge by searching for closed-form mathematical expressions that explain the observed data. Most existing SR approaches rely on genetic programming to explore the space of expression trees, whose hierarchical structure limits the reuse of intermediate computations while making the search space highly combinatorial. In this work, we propose *DragonSR*, a symbolic regression framework that formulates equation discovery as the optimization of directed acyclic graphs (DAGs). By combining graph-based architecture search with symbolic operators, DragonSR enables the reuse of intermediate symbolic computations while jointly optimizing graph topology, operator selection, and multi-channel symbolic representations within a unified optimization framework. The proposed approach is evaluated on a diverse collection of benchmark problems, including classical physical laws (e.g., Newtonian gravitation, Kepler’s law, and the ideal gas law), the Nguyen symbolic regression benchmarks and remote sensing spectral indices (e.g., NDVI, BSI, and NIRv). Experimental results demonstrate the ability of DragonSR to recover compact and interpretable symbolic expressions on a diverse collection of benchmark problems, highlighting the potential of DAG-based symbolic regression for scientific discovery and remote sensing.

1 Introduction

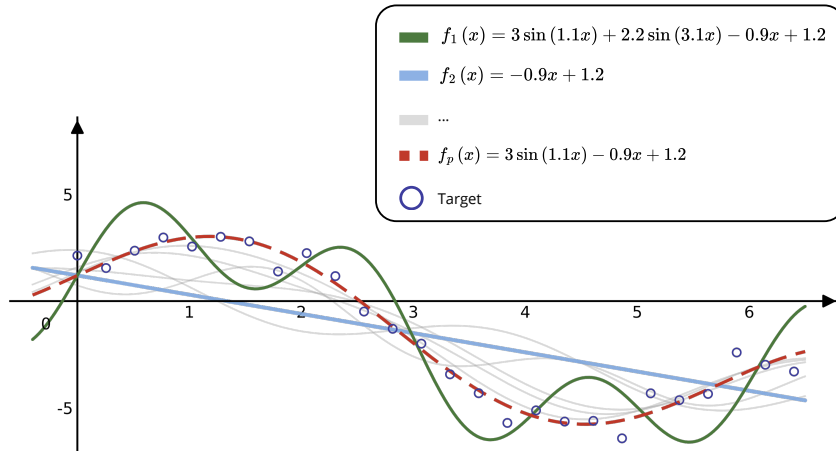


Figure 1: Illustration of symbolic regression. Starting from observed data points (blue), the search progressively generates candidate functions $f_1, f_2, \dots, f_p$ until recovering an analytical expression that accurately describes the underlying relationship.

Symbolic regression (SR) aims to recover analytical mathematical expressions directly from data while preserving interpretability. Unlike conventional regression methods, which optimize the parameters of a predefined model, symbolic regression simultaneously searches for both the structure of an equation and its numerical coefficients. By expressing solutions as combinations of elementary mathematical operators, symbolic regression provides compact and human-readable models that can reveal the underlying mechanisms governing the observed data. Consequently, symbolic regression has become an important tool for scientific discovery, particularly in domains where interpretability is as important as predictive performance, such as physics, chemistry, and remote sensing [1, 2, 3].

Recovering symbolic expressions is nevertheless a challenging combinatorial optimization problem due to the enormous search space induced by possible mathematical structures. Classical approaches based on genetic programming remain among the most widely adopted solutions, but rely on stochastic evolutionary operators whose exploration may require a large number of function evaluations [1, 4]. More recent methods have addressed these limitations through deterministic grammar-guided enumeration [5], domain-specific heuristics [6], reinforcement learning [7], and, more recently, large language models that exploit mathematical priors acquired during pretraining [8, 9, 10]. Although these approaches have significantly improved symbolic regression, efficiently exploring the search space while maintaining expressive representations remains an open challenge.

Neural architecture search (NAS) has demonstrated that complex search spaces can be efficiently explored by optimizing directed acyclic graph (DAG) representations instead of handcrafted architectures. Inspired by this observation, this work investigates whether a NAS framework can be adapted to symbolic regression. The proposed approach builds upon DRAGON [11], a DAG-based neural architecture optimization framework, and reformulates equation discovery as a graph optimization problem. Mathematical expressions are represented as directed acyclic graphs whose nodes correspond to elementary operators and whose edges propagate intermediate symbolic features. This representation naturally supports reusable intermediate computations and extends traditional expression trees by allowing richer computational structures. Most evolutionary symbolic regression methods represent expressions as trees, where each node corresponds to an operator and each leaf to an input variable or constant. While straightforward, tree representations duplicate sub-expressions that appear multiple times, unnecessarily increasing the search space.

To facilitate the exploration of the symbolic search space, several modifications to the original DRAGON framework are introduced. First, the search space is redesigned to represent symbolic expressions through specialized mathematical operators. Second, a multi-channel graph representation is proposed, enabling multiple intermediate symbolic features to propagate simultaneously across the graph. Finally, ordinary least squares combinations are incorporated to efficiently estimate combinations of symbolic terms and simplify the search process.

The proposed framework is evaluated on three complementary benchmarks: the Nguyen symbolic regression benchmark, a collection of classical physics equations, and remote sensing spectral indices. Particular emphasis is placed on noisy and multi-feature datasets, reflecting practical scenarios encountered in Earth observation applications. Experimental results illustrate the potential of the proposed graph-based formulation for symbolic regression across a diverse collection of benchmark problems, including noisy settings.

2 Related Work

2.1 Symbolic Regression: Problem Statement

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, with $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \mathbb{R}$, where N denotes the number of observations and d the number of input features, the goal of symbolic regression is to recover a closed-form expression f that best explains the relationship between $\mathbf{x}$ and y . Formally, expressions are built compositionally from a set of primitive operators $\mathcal{O} = \{+, -, \times, \div, \sin, \cos, \exp, \log, \dots\}$ and a terminal set $\mathcal{T} = \{x_1, \dots, x_d\} \cup \mathbb{R}$ combining input variables and free constants. The pair $(\mathcal{O}, \mathcal{T})$, together with the composition rules allowed to combine them, defines a search space of admissible expressions $\mathcal{F}_{\mathcal{O}, \mathcal{T}}$. SR then amounts to solving

$$f^* = \arg \min_{f \in \mathcal{F}_{\mathcal{O}, \mathcal{T}}} \mathcal{L}(f, \mathcal{D}), \quad (1)$$

where $\mathcal{L} : \mathcal{F}_{\mathcal{O}, \mathcal{T}} \times \mathcal{D} \rightarrow \mathbb{R}$ denotes an objective function measuring the discrepancy between the symbolic expression and the observed data.

Two features distinguish this optimization problem from standard regression. First, the search is combinatorial and unbounded in dimension: $\mathcal{F}_{\mathcal{O}, \mathcal{T}}$ is not a fixed-size parametric family but a discrete space of expression structures, each with its own continuous constants to be fitted. Second, the objective in (1) does not by itself guarantee interpretability or physical validity, which is why most methods also introduce structural priors on $\mathcal{F}_{\mathcal{O}, \mathcal{T}}$, as discussed below.

Physics-informed symbolic regression. When $\mathcal{D}$ originates from a physical system, the search space can be substantially constrained by exploiting domain knowledge, making the problem more tractable and its solutions more likely to be physically meaningful. Common priors include dimensional homogeneity (units of $f(\mathbf{x})$ must match those of y), symmetry and invariance properties, smoothness, low-order polynomial structure, and separability of the target function into simpler sub-problems. AI Feynman formalizes this idea by using such heuristics to recursively decompose a physics-inspired regression problem into a hierarchy of simpler ones [6], and its extension further exploits graph modularity in the same spirit [12]. These methods were evaluated on datasets of equations drawn from the *Feynman Lectures on Physics*, and similar principles now motivate a large share of SR applications to scientific discovery, where the objective is not only to fit $\mathcal{D}$ but to recover a law consistent with known physics.

A symbolic regression problem is defined by three components: (1) the search space $\mathcal{F}_{\mathcal{O}, \mathcal{T}}$ of admissible expressions, defined by the operator set $\mathcal{O}$, terminal set $\mathcal{T}$, and composition rules; (2) the search algorithm that explores this space; and (3) the objective function that evaluates candidate expressions. The following subsections review how existing methods approach each of these components.

2.2 State of the Art

This section surveys the main families of symbolic regression methods for a general machine learning audience. Rather than providing an exhaustive catalog, we highlight common design choices – expression representation (trees vs. graphs), search strategy (evolutionary, deterministic, or neural-guided), and objective function – and discuss how these choices affect the trade-off between expressivity and search efficiency. Symbolic regression emerged as a useful method for inferring mathematical expressions from data in the early 1970s with [13]. Expression trees were first explored in Pat Langley’s work with the BACON system and its extensions [14, 15, 16, 17]. Later, the use of genetic algorithms was introduced by Koza in genetic programming [1]. And symbolic regression reached wider adoption building on [18], with the development of EUREQA [2, 19] and HeuristicLab [20].

In the following, we review the main families of approaches to symbolic regression.

Genetic Programming. Building upon Koza’s genetic programming, numerous extensions of classical GP have been proposed [4]. Among modern implementations, `gplearn` provides a Python framework for genetic programming and symbolic regression, with a scikit-learn-compatible interface and a tree-based evolutionary search strategy [21]. Later, Cranmer *et al.* introduced PySR, an evolutionary symbolic regression framework combining parallel populations, Pareto optimization, and a Julia backend for efficient large-scale search. On a comprehensive benchmark suite containing equations from physics, the authors report state-of-the-art performance, outperforming several existing methods including Operon [22] and Deep Symbolic Regression (DSR) [3]. Despite their success, genetic programming methods remain inherently stochastic, relying on mutation and crossover operators to explore the search space. However, tree-based representations duplicate identical sub-expressions instead of sharing them, which unnecessarily increases both the search space and the computational cost of evaluating candidate expressions.

A parallel line of work has questioned the tree representation itself. Cartesian Genetic Programming (CGP) [23] encodes programs as directed, indexed graphs of nodes rather than trees, allowing sub-expressions to be reused across the program and mitigating the bloat commonly observed in tree-based GP. Positional CGP further evolves the position of each node in the graph, enabling new genetic operators while preserving this graph-based encoding [24]. Beyond evolutionary search, Kahlmeyer *et al.* [25] recently proposed an unbiased, systematic search over DAG-encoded expressions of bounded size, from which the variable augmentation procedure used in Section 3.3 is adapted. This line of work shows that representing programs as graphs, rather than trees, is a long-standing idea in evolutionary computation, and motivates the graph-based search space adopted in the present paper, though pursued here through a deterministic, DAG-optimization framework rather than an evolutionary one.

Deterministic and Enumerative Search. Prioritized Grammar Enumeration replaces stochastic evolutionary search with a deterministic grammar-guided exploration, where candidate expressions are systematically generated from production rules and prioritized according to Pareto optimality [5]. Fast Function Extraction (FFX) similarly departs from stochastic search, using a deterministic pathwise regularization procedure over a large basis of candidate functional forms [26]. AI Feynman leverages a similar search space simplification strategy by incorporating heuristics inspired by physicists’ intuition, as discussed in Section 2.1 [6].

Neural-guided search. Symbolic regression also draws on deep neural networks to address the interpretability problem of these models. Petersen *et al.* introduced a reinforcement learning method in which expression trees are generated node by node by a recurrent neural network, trained with policy gradient methods to maximize the quality of the generated expressions, replacing handcrafted search heuristics with a learned search policy [7]. Furthermore, these methods still generate expression trees and therefore do not address the structural redundancy inherent to tree representations.

LLM-guided search. A recent endeavor in symbolic regression is LLM-guided search. Rather than relying solely on evolutionary operators or reinforcement learning, these methods exploit the prior mathematical knowledge acquired during large-scale pretraining to generate, rank, or refine candidate expressions. Early work trained transformers from scratch on large synthetic corpora of expressions [8], while more recent methods directly leverage general-purpose pretrained LLMs [9, 10]. Another emerging direction leverages vision models to infer structural priors directly from function plots, providing guidance for the symbolic search [27]. These approaches, however, rely on pretrained mathematical knowledge rather than an explicit exploration of the symbolic search space.

2.3 Evaluation criteria and benchmarks.

Evaluating an SR method is not limited to raw predictive accuracy on $\mathcal{D}$. Four criteria are commonly considered jointly: (i) *exact recovery*, i.e., whether the retrieved expression is symbolically equivalent to the ground-truth formula, when one is known; (ii) *generalization accuracy* on held-out data, typically measured via R^2 or normalized MSE; (iii) *feature selection*, i.e., the ability to identify the relevant subset of input variables while discarding spurious or noisy ones; and (iv) *robustness to noise*, since real-world measurements are rarely noise-free and a method should degrade gracefully rather than overfit spurious structure.

To assess these criteria consistently, several benchmark suites have been proposed. The Nguyen benchmark, introduced alongside a semantically-based crossover operator for GP, consists of a set of low-dimensional polynomial and transcendental target functions and remains one of the most widely used SR benchmarks [28]. The Livermore benchmark set, introduced by the Lawrence Livermore National Laboratory team behind Deep Symbolic Regression, extends this idea with a larger and more diverse set of expressions used to evaluate neural-guided and hybrid GP methods [29]. The Feynman benchmark, drawn from equations of the *Feynman Lectures on Physics*, targets the physics-inspired setting discussed in Section 2.1 [6]. More recently, SRBench was proposed as a large-scale, standardized benchmark built on top of PMLB, comparing over a dozen SR methods on both synthetic (with known ground truth) and real-world (black-box) problems [30]. Together, these benchmarks provide the standard testbeds against which our approach is evaluated in Section 4.

2.4 From AutoML to the DRAGON Framework

Framed as in Section 2.1, symbolic regression can be viewed as an instance of Automated Machine Learning (AutoML): rather than tuning the hyperparameters of a fixed model family, SR searches directly over a space of model *structures* $\mathcal{F}_{\mathcal{O},\mathcal{T}}$ [31]. This connection is most explicit in Neural Architecture Search (NAS), the AutoML subfield concerned with automatically discovering neural network structures, which we introduce next as the basis of our own search procedure.

Beyond symbolic regression, graph-based optimization has become a central paradigm in NAS, where neural networks are represented as directed acyclic graphs (DAGs). Among these methods, DRAGON formulates architecture search as the joint optimization of a neural architecture α , a set of hyperparameters $\lambda(\alpha)$, and the corresponding trainable weights $\Theta(\alpha, \lambda)$ [11]. In this framework, the graph topology determines the computational structure, while node operators and hyperparameters define the family of admissible neural architectures. The search is therefore carried out jointly over both the graph structure and its associated parameters within a unified optimization framework.

The present work adapts this graph-based optimization paradigm to symbolic regression. Rather than representing expressions as trees, DragonSR formulates symbolic regression as the optimization of a directed acyclic computational graph whose nodes correspond to symbolic operators. This representation simultaneously enables the reuse of intermediate symbolic computations and the optimization of graph topology through the DRAGON framework, providing an alternative to both evolutionary graph search and classical expression trees.

Table 1: Comparison of the main families of symbolic regression methods.

Family	Search	Strengths	Limitations
Tree-based GP [3, 21]	Evolutionary	Flexible, expressive	Stochastic, many evaluations
Graph-based SR [23, 24, 25]	Evolutionary or exhaustive	Reusable subexpressions, less bloat	Evolved topology or bounded expression size
Deterministic Search [5, 6, 26]	Grammar / heuristics	Systematic, reproducible	Heuristic-dependent
Neural-guided [7]	Reinforcement learning	Learned search policy, adaptive	Costly, high-variance training
LLM-guided [8, 9, 10]	Foundation models	Strong mathematical priors	Hallucinations, pretrained knowledge dependent
DragonSR	Bandit-driven DAG optimization [32]	Shared subgraphs, multi-channel outputs	Stochastic search, many hyperparameters

Table 1 highlights a common trade-off among existing symbolic regression methods. Evolutionary approaches explore highly expressive search spaces but rely on expensive stochastic optimization. Deterministic methods improve reproducibility at the cost of strong heuristics, while neural- and LLM-guided methods introduce powerful priors but require costly training or pretrained knowledge. DragonSR combines deterministic graph optimization with a reusable DAG representation, while further extending the search space through multi-channel symbolic representations introduced in the next section.

3 Methods

The objective of DragonSR is to solve the symbolic regression problem introduced in Section 2 by identifying a symbolic function that accurately explains the relationship between the input variables and the target while remaining compact and interpretable. Rather than exploring the space of symbolic expressions as binary trees, DragonSR formulates this problem as the optimization of a directed acyclic graph (DAG), whose topology, primitive operators, and associated parameters jointly define a candidate symbolic expression.

The proposed methodology is developed progressively. We first formalize the optimization problem and define the corresponding search spaces. We then introduce the graph representation used to encode symbolic expressions and describe how candidate architectures are constructed. Building upon this representation, we present the feature selection mechanism and the multi-channel optimization strategy, which constitute the main extensions of DragonSR over classical symbolic regression approaches. We subsequently describe the mechanisms introduced to improve robustness in the presence of noisy observations before deriving the final optimization objective. Finally, the complete search procedure is summarized as pseudocode, providing an overview of the entire optimization pipeline.

3.1 Optimization problem

Recall the dataset $\mathcal{D}$ and the search space $\mathcal{F}_{\mathcal{O},\mathcal{T}}$ defined in Section 2.1. Let $d \in \mathbb{N}$ denote the number of input variables (features) and $N \in \mathbb{N}$ the number of observations. A dataset is defined as a pair (X, y) , where $X = (x_1, \dots, x_d) \in \mathbb{R}^{N \times d}$ is the input matrix whose columns correspond to the d input variables and $y \in \mathbb{R}^N$ is the target vector. Accordingly, the dataset space is $\mathcal{D} = \{(X, y) \mid X \in \mathbb{R}^{N \times d}, y \in \mathbb{R}^N\}$. Let $\mathcal{F} \subset \mathbb{R}^{\mathbb{R}^d}$ denote the search space of symbolic functions mapping the d input variables to a real-valued output. Any element of $\mathcal{F}$ is characterized by both

its symbolic structure and the parameters associated with its constituent operators. Accordingly, DragonSR decomposes the optimization problem into two complementary search spaces: the architecture search space $\mathcal{A}$, which specifies the arrangement of symbolic operators together with the discrete choices associated with them, and the parameter space $\mathcal{P}$, which contains the continuous learnable parameters of these operators (e.g., constants, exponents).

In the current implementation, the dataset space is restricted to real-valued tensors, and all primitive operators are applied element-wise. The proposed formulation could be extended to structured data types, such as tensors of matrices or other algebraic objects, provided that the primitive operators are well defined on the underlying domain. This would amount to replacing the domain $(\mathbb{R})$ in the definition of $\mathcal{D}$ by a more general algebraic structure while leaving the optimization framework unchanged. DragonSR inherits the graph optimization framework of DRAGON and therefore remains compatible with mixed search spaces containing both symbolic operators and neural modules. Such a unified formulation could naturally accommodate hybrid models in which interpretable symbolic expressions capture the analytical components of a phenomenon, while neural modules approximate highly nonlinear or noisy residual behaviors that are difficult to express analytically. This would enable symbolic and deep learning components to be optimized jointly within a single graph-based search process. The present work, however, focuses exclusively on symbolic equation discovery.

Having defined the search space, the optimization problem can now be formalized. An objective function ℓ is defined to quantify the search error, introduced later in Section 3.4:

$$\begin{aligned}\ell : \mathcal{F} \times \mathcal{D} &\longrightarrow \mathbb{R} \\ (f, (X, y)) &\longmapsto L(y, f(X))\end{aligned}$$

where $L : \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}$ denotes the loss function.

For a given architecture $\alpha \in \mathcal{A}$, the corresponding parameters $p \in \mathcal{P}$ are optimized as follows:

$$p^\star \in \arg \min_{p \in \mathcal{P}} \ell(f_{\alpha,p}, (X, y)).$$

The set of optimal symbolic functions induced by the architecture α is then defined as

$$\left\{ f_{\alpha,p} \mid p \in \arg \min_{p' \in \mathcal{P}} \ell(f_{\alpha,p'}, (X, y)) \right\}.$$

Therefore, the overall optimization objective is to identify

$$\alpha^\star, p^\star \in \arg \min_{\alpha \in \mathcal{A}, p \in \mathcal{P}} \ell(f_{\alpha,p}, (X, y)).$$

While the optimization procedure minimizes a prediction loss, the ultimate goal of symbolic regression is to recover the underlying symbolic expression that generated the observed data.

As an illustrative example, the Figure 2 presents two possible architectures along with their induced functions and optimized parameters, for a simple two-dimensional problem in $\mathbb{R}^2$ with two input features.

3.2 Architecture of the search space

Unlike binary trees, DAGs allow arbitrary connections between nodes, enabling the reuse of intermediate computations across multiple branches. This leads to more compact representations of symbolic expressions without increasing the depth of the computational graph. For instance,

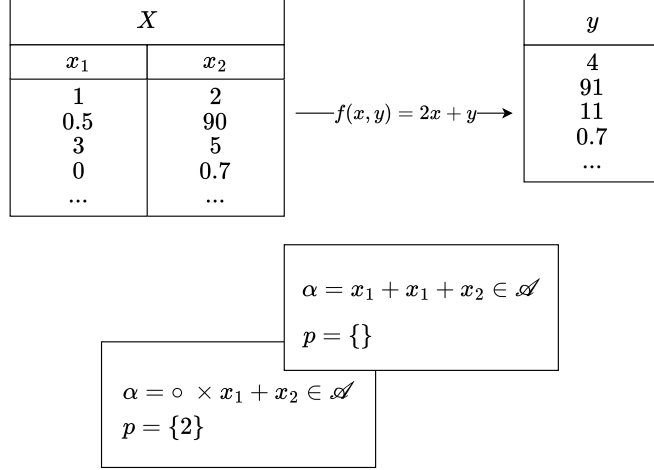


Figure 2: Illustration of two example architectures in $\mathbb{R}^2$, together with their induced symbolic functions and optimized parameters.

the expression $2x(x + x)$ requires a tree to recompute the subexpression $x + x$ independently at each occurrence, whereas a DAG stores and reuses a single copy of it. This example is borrowed from [33], who build on the same observation to introduce equality graphs (e-graphs), a structure that compactly stores expressions together with their equivalent forms so that the search avoids re-evaluating expressions it has already visited. In DragonSR, sub-expression sharing is not a memory of the search history but the representation of the candidate expression itself, extending classical expression trees with structural sharing of sub-expressions.

In DragonSR, this idea of structural sharing is enabled by the notion of combiner. Since each node may receive an arbitrary number of input connections, the latent representations of its parent nodes must first be combined before being passed to the node’s operation. DragonSR introduces two types of combiners: element-wise addition, element-wise multiplication. Since combiners operate channel-wise, their input representations must first be aligned to ensure compatible dimensions. Unlike conventional computational graphs with a fixed number of channels across all nodes, DragonSR allows the number of channels to vary throughout the graph. Therefore, channel alignment becomes an explicit component of the graph optimization process.

Because both the combiners and the operations themselves act element-wise, the number of channels produced at a node’s output is not fixed a priori but depends on that of its inputs; some operations, introduced in Section 3.4, are moreover explicitly designed to alter this channel dimension. This design choice plays a central role in the remainder of this work. Because the channel dimension is allowed to evolve throughout the graph, it becomes a search variable rather than a fixed architectural hyperparameter.

This has an important consequence for the resulting search space: beyond the compactness already afforded by structural sharing, the presence of a variable, operation-dependent channel dimension makes the space of representable architectures considerably richer than that of a classical expression tree — a gain in expressivity that comes at no additional cost in search budget.

Each DAG $\mathcal{G} = (V, E)$ is defined by its set of nodes $V = \{v_1, \dots, v_n\}$ and its set of edges $E \subseteq \{(v_i, v_j) \mid v_i, v_j \in V\}$. $\mathcal{G}$ can be represented by a sorted list L containing the graph nodes, and its adjacency matrix $M \in \{0, 1\}^{|L| \times |L|}$. A node v_k is composed of a combiner used to reconcile the entries in case of multiple inputs and an operation with its corresponding parameters (the

complete list of primitive operators is provided in Appendix C.). The input node is fixed to the **add** combiner and the **Identity** operator. This constraint is inherited from the adaptation of the original DRAGON architecture to the symbolic regression setting. An illustration of the graph representation and node definition is provided in Fig. 3.

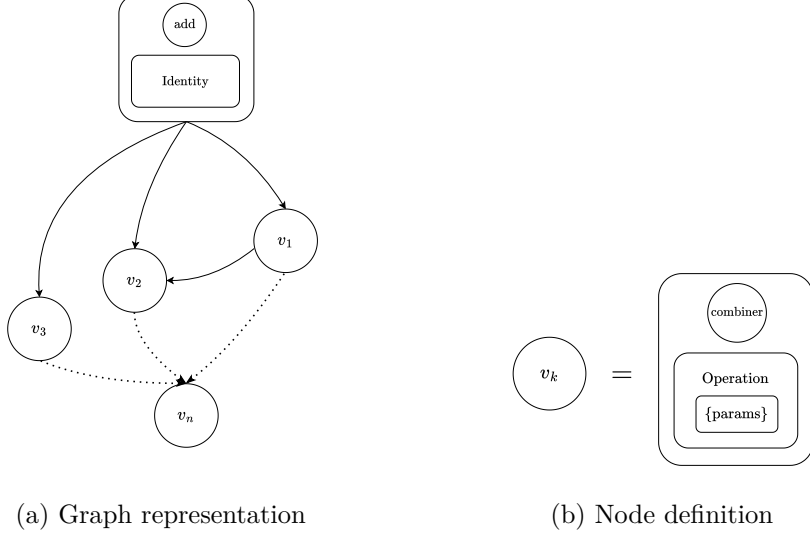


Figure 3: Overview of the graph structure and node definition.

Let $\alpha \in \mathcal{A}$ denote the architecture of a graph, and let $\mathcal{P}$ denote the space of continuous parameters associated with the primitive operators. Let $\mathcal{G}$ be the corresponding DAG. Its representation is given by $\mathcal{G} = (L, M)$, where $L = (v_0, v_1, \dots, v_n)$ is the ordered list of nodes, with v_0 the fixed input node, and $M \in \{0, 1\}^{|L| \times |L|}$ the corresponding adjacency matrix. Each node $v_k \in L$ is defined as $v_k = p_k \circ c_k$, where p_k denotes a primitive operator together with its associated optimizable parameters, and c_k the combiner acting on the outputs of the predecessor nodes.

The combiner c_k operates on the feature tensors S_{i_j} produced by the selected predecessor nodes and is defined as

$$c_k : (S_i)_{M_{i,k}=1} \mapsto \begin{cases} S_{i_1} \diamond \dots \diamond S_{i_\ell}, & \ell > 1, \\ S_{i_1}, & \ell = 1, \end{cases}$$

where $\{i_1, \dots, i_\ell\} = \{i \mid M_{i,k} = 1\}$ and $\diamond \in \{+, \times\}$. If the selected tensors have different dimensions, they are first padded to a common size before applying the combiner (more of this in appendix F).

And each primitive operator is defined by the triplet

$$p_k = (o_k, \eta_k, \theta_k),$$

where o_k denotes the operator type, η_k the operator-specific architectural parameters, and θ_k the associated learnable continuous parameters. Together, (o_k, η_k) are specified by the architecture α and optimized by the DRAGON search procedure, while $\theta_k \in \mathcal{P}$ is optimized using the selected continuous-parameter optimizer, either a dichotomy-based line search or gradient-based optimization, when such parameters are present.

Although a DAG only induces a partial order on its nodes, the vertices are assumed to be indexed according to a topological ordering. This convention is used in the recursive definition of the symbolic function induced by the graph, presented below.

Let S_k denote the output associated with the node v_k . The input node is initialized as $S_0 = X$, where $X \in \mathbb{R}^{N \times d}$ denotes the input dataset.

Accordingly, the outputs are defined recursively by

$$S_{k+1} = p_{k+1} \circ c_{k+1} \left((S_i)_{M_{i,k+1}=1} \right).$$

Therefore the symbolic function induced by the DAG $\mathcal{G}$ is defined by $f_{\mathcal{G}} : X \mapsto S_n$, where S_n denotes the output of the last node in the topological ordering.

An illustrative example of the symbolic function induced by an example DAG and its corresponding adjacency matrix is shown in Figure 4.

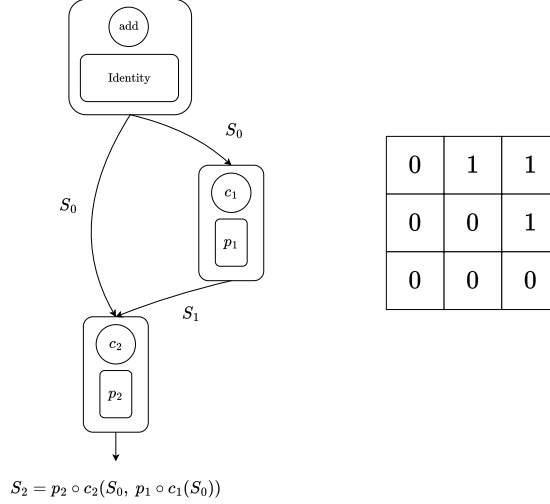


Figure 4: Illustration of a three-node DAG producing the final output function and its associated adjacency matrix representation

3.3 Features selection

Having introduced the search space and its output representation, the construction of the input feature set is now described. The method introduces variable augmentation, adapted from [25], to better exploit the multi-channel structure of the DAGs while simplifying the search process.

Given an input dataset $X = (x_1, x_2, \dots, x_d) \in \mathbb{R}^{N \times d}$, the method applies a set of primitive operators from $\mathcal{O}_{VA}$ (see Appendix D) to construct augmented features.

The framework first constructs an augmented feature space by exhaustively applying every operator in $\mathcal{O}_{VA}$ to the input variables. Unary operators are applied independently to each feature, while binary operators are evaluated for every pair of variables. Let $\mathcal{C}$ denote the resulting set of candidate features,

$$\mathcal{C} = \{g(x) \mid g \in \mathcal{O}_{VA}\}.$$

Each candidate feature is then evaluated independently according to its squared Pearson correlation R^2 with the target (see Appendix G).

The feature

$$c^* = \arg \max_{c \in \mathcal{C}} R^2(c, y)$$

is then selected as the best unary candidate. In parallel, all pairwise combinations of candidate features are evaluated using the binary operators of $\mathcal{O}_{\text{VA}}$, and the pair maximizing the same criterion is retained.

Furthermore, once the variable augmentation process is completed, an additional selection step is applied to the expanded feature set. Given a user-specified parameter n , the framework selects the top- n features based on their importance scores estimated using a non linear regression model trained on the augmented feature space. Both the input features and the target variable are normalized using standard score normalization prior to training. The model is configured with $n_{\text{estimators}}$ trees, a maximum depth of d , and a learning rate η . Feature importance scores are extracted from the trained model using the built-in gain-based importance metric. Let $I(c)$ denote the importance of feature c . The framework then selects the top- n features with highest $I(c)$ values, forming the final reduced feature set.

The overall initial feature selection pipeline is summarized in Figure 5 below, and instantiated on a concrete remote-sensing example in Appendix B..

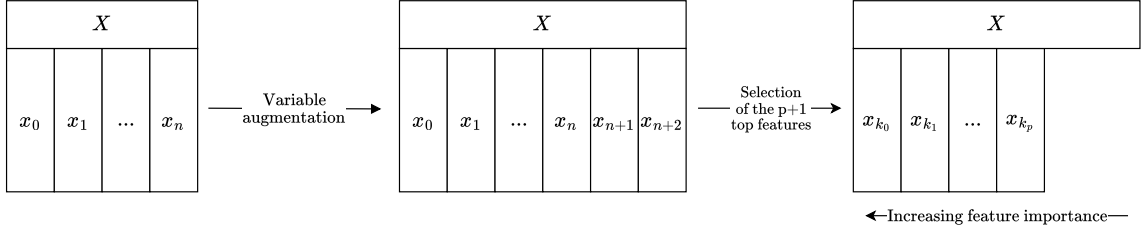


Figure 5: Initial feature selection pipeline combining operator-driven feature expansion and XGBoost-based ranking.

3.4 Multi-Channel Output Optimization

The symbolic function induced by a DAG was previously defined as $f_{\mathcal{G}} : X \mapsto S_n$, where S_n denotes the output of the last node v_n in the topological ordering. In the initial formulation, S_n is composed of m output channels,

$$S_n = (ch_1, \dots, ch_m),$$

where each channel $ch_i \in \mathbb{R}^N$ corresponds to a symbolic representation evaluated over the N observations of the dataset. Since all primitive operators act element-wise and combiners only perform element-wise aggregation after dimension matching through padding, the channel dimensionality is preserved during graph evaluation. Therefore, the number of output channels remains fixed unless explicitly modified by a dedicated channel operation.

This multi-channel representation directly impacts the optimization objective. Applying the regression loss to the graph output produces one loss value per output channel,

$$\ell(f_{\mathcal{G}}, y) = L(y, S_n) = (L(y, ch_1), \dots, L(y, ch_m)),$$

where ℓ denotes the regression loss introduced in Section 3.1. A scalar objective is therefore required to evaluate each candidate DAG during the search. In the present work, the graph loss is defined as

$$\ell(f_{\mathcal{G}}, y) = \min_{1 \leq i \leq m} L(y, ch_i).$$

While this formulation allows optimization of multi-channel representations, maintaining a fixed channel structure restricts the range of architectures explored during the search. To

overcome this limitation, two additional operators acting directly on the channel dimension are introduced. Unlike the symbolic primitives presented previously, these operators modify the channel representation itself rather than only transforming the symbolic expressions carried by each channel.

The first operator, **SelectFeatures**, performs learned feature selection. Given an input tensor $(x_1, \dots, x_p)$, it returns a sub-tensor $(x_{i_1}, \dots, x_{i_m})$, where the indices $\{i_1, \dots, i_m\}$ are sampled according to a probability distribution induced by a prior feature ranking. The selection probability is proportional to the importance scores: $p(i_j) \propto I(i_j)$, where $I(i_j)$ denotes the importance of the j -th feature from the XGBoost model introduced in Section 3.3.

The second operator, **ChannelBoost**, performs feature augmentation by constructing a new channel from selected interactions between input features. Given a tensor $(x_1, \dots, x_p)$, it combines boundary features (x_1 and x_p) through a learnable choice among elementary operations: addition, multiplication, subtraction, or division. The resulting feature is concatenated to the original representation, increasing the expressive capacity of the channel space.

Together, these operators introduce a complementary optimization process acting on the channel representation in parallel with symbolic expression discovery. In addition to searching over symbolic operators and graph topologies, the framework optimizes the feature representation by selecting, restructuring, and augmenting intermediate channels according to their predictive relevance with respect to the target variable y . Their impact on symbolic equation discovery is evaluated experimentally in Section 4.

A complete worked example combining the preprocessing pipeline of Section 3.3 and the channel operators introduced above is provided in Appendix B: starting from raw Sentinel-2 bands, it traces the variable augmentation, the XGBoost-based selection of the top features, and the evaluation of a candidate DAG in which **SelectFeatures** and element-wise primitives progressively transform the channel representation into the final symbolic output.

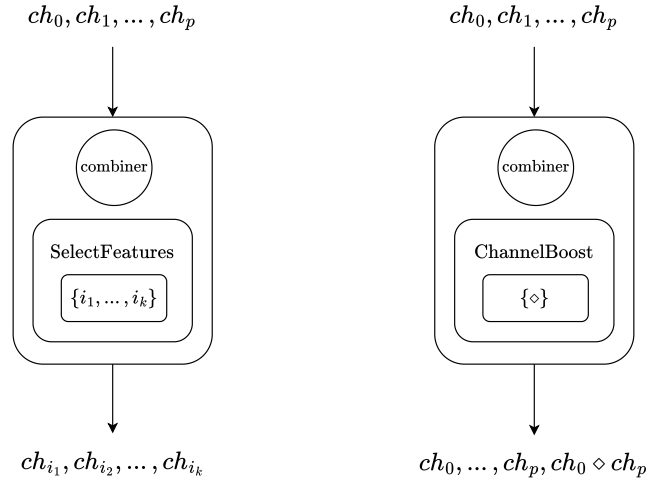


Figure 6: Overview of the **SelectFeatures** and **ChannelBoost** operators

A further extension consists in replacing the channel-wise objective by a learned aggregation of the output channels. Rather than evaluating each channel independently, the multi-channel output of the graph is first projected onto a single scalar representation before computing the search objective.

Let $g : \mathbb{R}^{N \times m} \rightarrow \mathbb{R}^N$, where N denotes the number of training samples and m the number of output channels. Given the final graph output $S_n = (ch_1, \dots, ch_m)$, the aggregation function g

combines the channels into a single prediction $\hat{y} = g(ch_1, \dots, ch_m)$.

The search objective then becomes

$$\ell(f_{\mathcal{G}}, y) = L(y, g(ch_1, \dots, ch_m)),$$

where L denotes the regression loss introduced in Section 3.1.

This formulation extends the search space beyond symbolic expressions represented by individual channels. Each channel is allowed to encode a partial symbolic component of the target equation, while the final prediction is obtained through their aggregation. Consequently, the evolutionary search is no longer restricted to discovering a complete symbolic expression within a single computation path, but may instead distribute the representation across multiple channels before combining them into the final model.

Several strategies were considered for instantiating the aggregation function g .

Sparse linear aggregation. The simplest aggregation strategy computes a sparse linear combination of the output channels,

$$\hat{y} = \sum_{j \in \mathcal{A}} w_j ch_j + b,$$

where $\mathcal{A} \subseteq \{1, \dots, p\}$ denotes the active set of channels retained in the final combination. The active set is obtained through an iterative thresholding procedure. Let $\mathcal{A}^{(0)} = \{1, \dots, p\}$. At iteration t , an ordinary least squares regression is fitted on the channels currently in $\mathcal{A}^{(t)}$,

$$(w^{(t)}, b^{(t)}) = \arg \min_{w, b} \left\| y - \sum_{j \in \mathcal{A}^{(t)}} w_j ch_j - b \right\|_2^2.$$

Channels are then eliminated according to an adaptive threshold,

$$\tau^{(t)} = \max \left(\rho \cdot \max_{j \in \mathcal{A}^{(t)}} |w_j^{(t)}|, \varepsilon \right), \quad \mathcal{A}^{(t+1)} = \left\{ j \in \mathcal{A}^{(t)} : |w_j^{(t)}| \geq \tau^{(t)} \right\},$$

where ρ is a relative threshold (set to 10^{-3}) and ε is a small absolute floor (set to 10^{-8}) preventing degenerate thresholds when all weights are close to zero. The procedure stops as soon as no channel is eliminated, i.e. $\mathcal{A}^{(t+1)} = \mathcal{A}^{(t)}$, or after a fixed number of iterations. The final weights are obtained by refitting the ordinary least square regression on the resulting active set $\mathcal{A}$. This leads to a sparse linear combination of the output channels while avoiding the combinatorial cost of an explicit subset-selection search.

Nested aggregation. Nested aggregation extends ordinary least squares by performing the regression in a transformed output space. Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a bijective link function selected from a predefined family (e.g., identity, logarithm, square root, inverse; see Appendix D), and let u_j denote a unary transformation independently selected for each output channel from the same set. The aggregated prediction is defined as

$$\hat{y} = f^{-1} \left(\sum_{j=1}^p w_j u_j(ch_j) + b \right).$$

For each candidate graph, the target is first transformed according to $\tilde{y} = f(y)$. For every channel, the unary transformation u_j minimizing the regression error with respect to y_t is then

selected, and following the procedure described above, a sparse linear aggregation is fitted in the transformed space. The inverse link f^{-1} is finally applied to recover the prediction in the original output space. This procedure captures simple nonlinear relationships between the output channels and the target variable while preserving the computational efficiency of linear regression.

Polynomial Rational Aggregation. An alternative aggregation strategy models the final prediction as a rational function of the output channels. Let

$$\Phi(ch) = \{\phi_1(ch), \dots, \phi_M(ch)\}$$

denote a polynomial basis constructed from the channel outputs, where the current implementation includes monomials of degree one and two. The aggregated prediction is defined as

$$\hat{y} = \frac{a_0 + \sum_{m=1}^M a_m \phi_m(ch)}{1 + \sum_{m=1}^M b_m \phi_m(ch)}.$$

Direct optimization of this rational model is nonlinear. Instead, the parameters are estimated by exploiting the identity satisfied by the exact solution:

$$\hat{y} \left(1 + \sum_{m=1}^M b_m \phi_m \right) = a_0 + \sum_{m=1}^M a_m \phi_m.$$

During training, the unknown prediction $\hat{y}$ is replaced by the observed target y , yielding the approximation

$$y \left(1 + \sum_{m=1}^M b_m \phi_m \right) \approx a_0 + \sum_{m=1}^M a_m \phi_m.$$

Rearranging the terms leads to

$$y \approx a_0 + \sum_{m=1}^M a_m \phi_m - \sum_{m=1}^M b_m y \phi_m,$$

which is linear in the unknown coefficients $a_0, \dots, a_M, b_1, \dots, b_M$. The resulting least-squares problem therefore uses a design matrix composed of the polynomial basis functions ϕ_m together with the interaction terms $-y\phi_m$. This reformulation enables rational combinations of the symbolic channels to be estimated using standard linear regression, without requiring nonlinear optimization.

These three aggregation strategies are not restricted to a single fixed choice: they can be combined with the single-channel objective defined previously, $\ell(f_{\mathcal{G}}, y) = \min_{1 \leq i \leq m} L(y, ch_i)$, to guide the search in different ways. Each aggregation strategy g produces a scalar prediction $\hat{y}$ together with its own loss $L(y, \hat{y})$, so that, together with the single-channel objective, up to four candidate loss values are available for a given DAG. Depending on the configuration, the search can be guided either by the minimum loss over the four candidates or by a single selected objective, such as the single-channel objective or a specific aggregation strategy.

3.5 Noise handling: subsampling and surrogate modeling

A key objective of DragonSR is to recover interpretable symbolic expressions from noisy observations, as encountered in most real-world scientific datasets. To evaluate the robustness of the proposed approach, synthetic Gaussian noise is added to the target variable while leaving the input variables unchanged, following the common evaluation protocol adopted in symbolic regression benchmarks [7, 30]. Given the noiseless target y , the noisy target is generated as

$$y_{\text{noise}} = y + \varepsilon, \quad \varepsilon \sim \mathcal{N}\left(0, \sigma_y^2 \lambda^2\right), \quad (2)$$

where $\sigma_y = \text{std}(y)$ denotes the standard deviation of the target and λ is the prescribed relative noise level.

Rather than directly optimizing symbolic expressions on the noisy target, two complementary denoising strategies are investigated.

Stochastic subsampling. At each optimization step, the regression loss is evaluated on a randomly sampled subset of the available observations. Formally, let $I_t \subset \{1, \dots, p\}$ denote the subset sampled at iteration t . The optimization objective becomes

$$\ell_t(f_{\mathcal{G}}, y_{\text{noise}, I_t}) = L(f_{\mathcal{G}}(X_{I_t}), y_{\text{noise}, I_t}),$$

where X_{I_t} and y_{noise, I_t} denote the corresponding subsets of the data. Since the sampled observations vary from one iteration to the next, the optimization becomes less sensitive to individual noisy measurements while preserving the overall structure of the underlying function.

Surrogate model. The second strategy consists of fitting a non-linear regression model on the noisy observations prior to symbolic regression. Let $h : \mathbb{R}^n \rightarrow \mathbb{R}$ denote the surrogate model. The original target is replaced by the surrogate prediction

$$\hat{y} = h(X),$$

and DragonSR is subsequently optimized using the surrogate target $\hat{y}$ instead of y_{noise} . The surrogate acts as a smoothing function, filtering high-frequency noise while preserving the dominant functional relationships to be recovered by symbolic regression.

3.6 Search Objective

The optimization objective ℓ introduced in Section 3.1 is defined from a loss function $L : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$. DragonSR is agnostic to the choice of L and supports any differentiable or non-differentiable regression loss. The current implementation includes standard objectives such as the mean squared error (MSE), mean absolute error (MAE), and Huber loss, while users may also define custom loss functions.

Beyond these classical regression losses, DragonSR also supports Pearson correlation as the search objective. Haut *et al.* [34] showed that correlation-based fitness discovers symbolic expressions in fewer generations than RMSE on several benchmark problems. Similar behavior is observed in the experiments presented in Section 4, where correlation generally leads to faster convergence and improved symbolic recovery.

When Pearson correlation is selected, the loss function is defined as

$$L_{\text{corr}}(y, f_{\mathcal{G}}(X)) = 1 - \rho(y, f_{\mathcal{G}}(X)),$$

where ρ denotes the Pearson correlation coefficient (see Appendix G).

Unlike MSE or MAE, Pearson correlation is invariant under affine transformations of the predictions. Consequently, two symbolic expressions differing only by a scaling factor and an offset receive the same loss value. To recover the final symbolic expression, the graph output is therefore aligned to the target through the affine transformation

$$\hat{y} = \alpha f_{\mathcal{G}}(X) + \beta,$$

where

$$(\alpha, \beta) \in \arg \min_{a, b \in \mathbb{R}} \|a f_{\mathcal{G}}(X) + b - y\|_2^2.$$

The aligned prediction is subsequently used for evaluation, yielding the final symbolic expression

$$\hat{f}_{\mathcal{G}} = \alpha f_{\mathcal{G}} + \beta.$$

This alignment is only performed when Pearson correlation is used as the search loss. For scale-sensitive losses such as MSE, MAE, or Huber, the graph output is evaluated directly.

In addition to the regression loss, DragonSR optionally incorporates a structural regularization term that discourages excessive compositions of functions from the same family. While symbolic regression naturally favors compact expressions, the search process may produce unnecessarily deep nested operators such as $\sin(\sin(x))$, $\exp(\exp(x))$, or repeated logarithmic compositions. Such expressions generally increase symbolic complexity without improving predictive performance and reduce interpretability.

When enabled, the optimization objective becomes

$$\ell(f_{\mathcal{G}}, y) = L(y, f_{\mathcal{G}}) + \lambda_{\text{comp}} C(\mathcal{G}),$$

where λ_{comp} is a regularization coefficient and $C(\mathcal{G})$ denotes the composition complexity penalty.

For each function node belonging to a predefined operator family (e.g., trigonometric, exponential, or logarithmic functions), the algorithm computes its nesting depth d_i , defined as the length of the longest chain of successive operators from the same family ending at node i . Only repeated compositions ($d_i \geq 2$) are penalized, according to

$$C(\mathcal{G}) = \sum_{i: d_i \geq 2} (b^{d_i-1} - 1),$$

where $b > 1$ controls the exponential growth of the penalty. Consequently, shallow compositions incur only a small additional cost, whereas deeply nested expressions are penalized much more strongly. This regularization encourages the discovery of simpler symbolic expressions while preserving sufficient flexibility to represent complex nonlinear relationships.

3.7 Pseudo-code of the complete search procedure

Algorithm 1 summarizes the complete DragonSR optimization pipeline, from data preprocessing to symbolic expression recovery. The preprocessing stage applies the optional denoising, variable

augmentation, and feature selection procedures introduced in the previous sections before launching the symbolic search.

The search itself is performed by the DRAGON optimizer, which incrementally explores graph architectures of increasing complexity. At each complexity level, candidate directed acyclic graphs are proposed by the Mutant-UCB search procedure [32] and evaluated through the procedure described in Algorithm 2.

Algorithm 1 DragonSR

Require: Dataset (X, y) ; search budget MAX_COMPLEXITY, K_INIT, T_PER_LEVEL

Ensure: Symbolic expression f^*

Preprocessing

```
1: if denoiser then
2:    $y \leftarrow \text{DENOISE}(X, y)$ 
3: end if
4: if var_aug then
5:    $X \leftarrow \text{AUGMENTFEATURES}(X)$ 
6: end if
7:  $X \leftarrow \text{TOPFEATURES}(X, y, \text{N\_TOP\_FEATURES})$ 
```

Search

```
8:  $\ell^* \leftarrow +\infty$ 
9:  $\mathcal{G}^* \leftarrow \emptyset$ 
10: total_iters  $\leftarrow 0$ 
11: for  $c = 1$  to MAX_COMPLEXITY do
12:    $r \leftarrow \text{N\_ITERATIONS} - \text{total\_iters}$ 
13:   if  $r \leq 0$  then
14:     break ▷ global evaluation budget exhausted
15:   end if
16:    $T \leftarrow |\text{current population}| + \min(\text{T\_PER\_LEVEL}, r)$ 
17:    $(\ell, \mathcal{G}) \leftarrow \text{MUTANTUCB}(\text{EVALUATE}, T, \text{K\_INIT}, c)$ 
18:   total_iters  $\leftarrow |\text{current population}|$ 
19:   if  $\ell < \ell^*$  then
20:      $\ell^* \leftarrow \ell$ 
21:      $\mathcal{G}^* \leftarrow \mathcal{G}$ 
22:   end if
23: end for
```

Formula extraction

```
24: return FORMULAOF( $\mathcal{G}^*$ )
```

Algorithm 2 EVALUATE($\mathcal{G}$)

Require: Candidate DAG $\mathcal{G}$; dataset (X, y)

Ensure: Search loss ℓ

```
1:  $f_{\mathcal{G}} \leftarrow \text{BUILD}(\mathcal{G})$ 
2: if  $f_{\mathcal{G}}$  has learnable parameters then
3:   Optimize them on  $(X, y)$  using the selected optimization method ▷ e.g., gradient descent
   or dichotomous search
4: end if
5:  $(X_{I_t}, y_{I_t}) \leftarrow \text{random subsample of } (X, y)$ 
6:  $S_n \leftarrow f_{\mathcal{G}}(X_{I_t})$  ▷ multi-channel output (Section 3.4)
7:  $\hat{y} \leftarrow g(S_n)$  ▷ aggregation
8: if no valid channel then
9:   return  $+\infty$ 
10: end if
11:  $\ell \leftarrow \text{SEARCHLOSS}(\hat{y}, y_{I_t}) + \text{COMPOSITIONPENALTY}(f_{\mathcal{G}})$ 
12: return  $\ell$ 
```

To improve the exploration of heterogeneous symbolic search spaces, DragonSR can execute several independent search streams in parallel. Rather than sharing the same operator library, each stream is assigned a predefined subset of primitive operators, thereby defining its own search space. The global computational budget is evenly partitioned among the streams, and each stream independently executes Algorithm 1 using its assigned operator set. Once all searches terminate, the symbolic expression achieving the lowest search loss is retained as the final solution.

Figure 7 illustrates the overall execution workflow. After the common preprocessing stage, the search is distributed across several independent workers, each operating on its own operator library. Every worker executes Algorithms 1 and 2 independently before the best recovered symbolic expression is selected according to the chosen search objective.

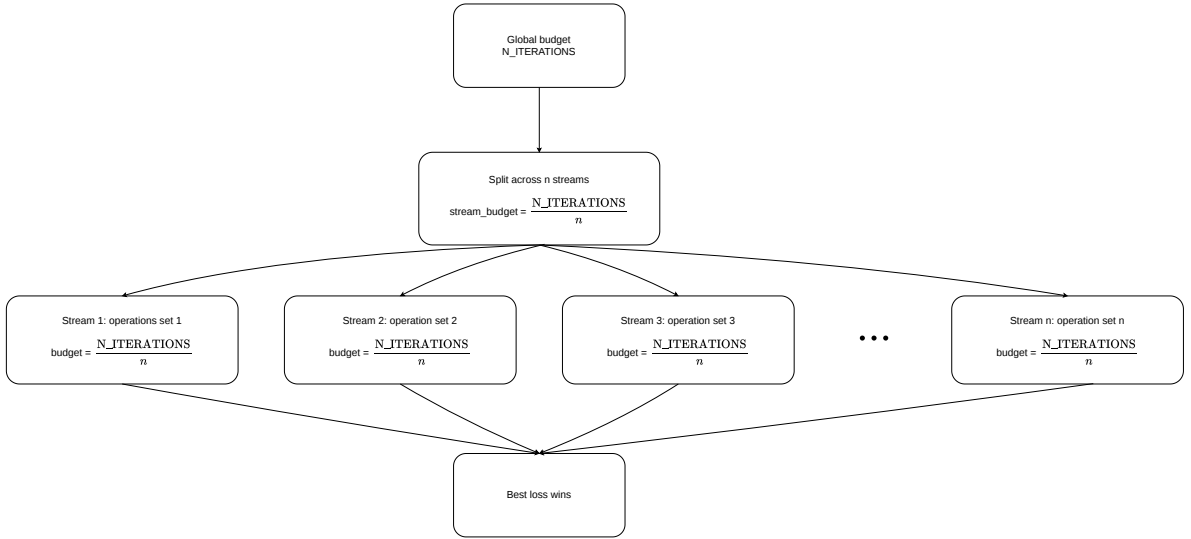


Figure 7: Budget and Operator Partitioning Across Streams

4 Experiments

4.1 Benchmark and Experimental Setup

We evaluate DragonSR on a benchmark composed of three categories of symbolic regression problems. The complete list of analytical expressions is provided in Appendix A.1.

The first category consists of the Nguyen benchmark equations, which have become a standard reference for evaluating symbolic regression algorithms on synthetic mathematical expressions. They provide a controlled setting for assessing the ability of an algorithm to recover polynomial, transcendental and multivariate relationships.

The second category contains analytical equations originating from classical physics, including laws from mechanics, thermodynamics, astrophysics and spectroscopy. These equations were selected to match those used in the PySR benchmark [3], allowing a direct comparison with one of the current state-of-the-art symbolic regression methods on scientifically relevant problems.

Finally, we introduce a third category composed of remote sensing indices derived from Sentinel-2 multispectral imagery. Unlike the previous benchmarks, these problems involve more than ten input variables and therefore evaluate the scalability of the search procedure to higher-dimensional feature spaces.

For all synthetic datasets (Nguyen and Physics), 6 000 observations are generated from the

analytical expressions using the sampling distributions reported in Appendix A.2. This choice differs from the traditional symbolic regression literature, where benchmarks are typically constructed from only a few dozen or a few hundred samples. While small datasets simplify the search problem, they are not representative of many scientific applications, where measurements are available in much larger quantities. Moreover, in the presence of measurement noise, increasing the number of observations does not necessarily simplify symbolic regression, as the optimization procedure must discriminate the underlying analytical relationship from an increasingly heterogeneous cloud of noisy samples.

To evaluate robustness to noisy observations, Gaussian noise is added only to the target variable according to (2). The relative noise level is controlled through $\epsilon \in \{0, 0.01, 0.05, 0.1\}$, corresponding respectively to noise-free data and increasing levels of perturbation. The input variables are left unchanged, following the experimental protocol commonly adopted in symbolic regression benchmarks.

The search spaces and computational budgets used for both DragonSR and PySR are summarized in Appendix E.1, while the complete DragonSR configurations are reported in Appendix E.2. Unless stated otherwise, DragonSR is executed for 10 000 iterations with a maximum graph complexity of 55, feature sampling, variable augmentation, parallel populations and random initialization enabled.

Feature pre-selection is performed using an XGBoost regressor [35], from which the highest-ranked variables according to feature importance are retained before the symbolic search. Continuous parameters introduced by symbolic operators are optimized through gradient descent using the Adam optimizer [36]. When denoising is enabled, the target values are first smoothed using a Gaussian Process Regressor with a Matérn kernel [37] before being provided to DragonSR. Furthermore stochastic subsampling is performed by evaluating the optimization objective on a random subset containing 10% of the available observations at each iteration.

4.2 Results

This section reports the symbolic recovery results obtained by PySR and DragonSR on the proposed benchmark. Each experiment was repeated over ten independent runs, and the reported metric corresponds to the number of runs for which the discovered expression was successfully recovered. A recovery is counted when the obtained expression is considered equivalent to the ground-truth equation after algebraic simplification and numerical verification.

For DragonSR, several configurations described in Appendix E.2 were evaluated. For each experimental setting, only the final candidate achieving the lowest search loss was retained, independently of the other expressions explored during the search process. No multi-objective selection criterion was applied, and the reported results therefore correspond to the best final solution according to the chosen optimization objective.

The complete benchmark results, including all individual runs reported in this table, are described in Appendix I, together with the corresponding dataset files and implementation resources.

Table 2 summarizes the recovery rates obtained across the three benchmark categories: Nguyen equations, physics-inspired equations, and remote sensing datasets. The first two columns indicate the benchmark category and dataset name, while the remaining columns report the number of successful recoveries obtained by PySR and DragonSR for the different noise levels. Both methods were evaluated using the same GPR-based denoising procedure to ensure a consistent comparison.

Table 2: Performance comparison between PySR and DragonSR under different additive noise levels $\epsilon \in \{0, 0.01, 0.05, 0.10\}$. DragonSR configurations are specified in Appendix E. Both methods use GPR denoising. Counts are by human judgment (formulas read and algebraically compared to ground truth).

Category	Dataset	PySR				DragonSR			
		0	0.01	0.05	0.10	0	0.01	0.05	0.10
Nguyen	N4	0/10	0/10	0/10	0/10	8/10	1/10	0/10	1/10
	N5	0/10	0/10	0/10	0/10	3*/10	3/10	1/10	1/10
	N6	0/10	0/10	0/10	0/10	0/10	6/10	5/10	4/10
	N7	0/10	0/10	0/10	0/10	4*/10	1/10	0/10	0/10
	N8	10/10*	0/10	0/10	0/10	9/10	10/10	9/10	8/10
	N9	0/10	0/10	0/10	0/10	10/10	9/10	8/10	8/10
	N10	10/10*	0/10	0/10	0/10	0/10	4/10	3/10	3/10
	N11	10/10*	0/10	0/10	0/10	0/10	1/10	0/10	1/10
	N12	6/10*	0/10	0/10	0/10	0/10	5/10*	4/10*	8/10*
	Hubble	9/10*	0/10	0/10	0/10	9/10	6/10	0/8	1/10*
	Newton	0/10	0/10	0/10	0/10	0/10	0/10	0/10	0/10
	Rydberg	0/10	0/10	0/10	0/10	0/10	7/10	6/10	8/10
Physics	Ideal Gas	1/10*	0/10	0/10	0/10	9/10	6/10	3/10	4/10
	Kepler 3rd	0/10	0/10	0/10	0/10	10/10*	3/10	4/10	1/10
	Bode's law	0/10	0/10	0/10	0/10	0/10	10/10*	0/10	0/10
	Schechter	0/10	0/10	0/10	0/10	0/10	0/10	4/10*	1/10*
	Leavitt	2/10*	0/10	0/10	0/10	10/10	6/10	7/10*	5/10*
	Planck's law	0/10	0/10	0/10	0/10	1/10*	1/10*	0/10	0/10
	NDVI	3/10*	0/10	0/10	0/10	0/10	4/10	1/10	2/10
	WI2015	0/10	0/10	0/10	0/10	0/10	0/10	0/10	0/10
Remote Sensing	AWEI_sh	8/10	0/10	0/10	0/10	10/10*	3/10*	1/10*	0/10
	BAI	0/10	0/10	0/10	0/10	0/10	0/10	0/10	0/10
	BSI	0/10	0/10	0/10	0/10	9*/10	6/10*	3/10*	9*/10
	EVI2	0/10	0/10	0/10	0/10	9/10	8/10*	3/10*	3/10*
	VARI	0/10	0/10	0/10	0/10	0/10	0/10	0/10	0/10
	SAVI	0/10	0/10	0/10	0/10	10/10	9/10*	10/10*	8/10*
	NIRv	1/10*	0/10	0/10	0/10	0/10	0/6	0/10	0/10

* The recovered expression was simplified by removing negligible terms (i.e., coefficients close to zero).

DragonSR consistently recovers symbolic expressions across all three benchmark categories, with robust performance across noise levels. On the Nguyen benchmark, DragonSR achieves perfect or near-perfect recovery on several targets at $\epsilon = 0$ (N8: 9/10, N9: 10/10) and maintains useful recovery rates up to 10% noise. PySR shows strong results at $\epsilon = 0$ on simpler targets (N8: 10/10*, N10: 10/10*, N11: 10/10*) but its recovery drops to zero at $\epsilon > 0$ across all Nguyen targets, suggesting that PySR's tree-based evolutionary search is less robust to noisy observations.

On physics-inspired benchmarks, DragonSR recovers Hubble's law (9/10), the ideal gas law (9/10), and Leavitt's law (10/10) at $\epsilon = 0$, and demonstrates graceful degradation under increasing noise. These results highlight the benefit of the multi-channel DAG representation, which allows the search to explore partial symbolic components via independent channels before combining them into the final expression. PySR achieves partial recovery on Hubble (9/10*) but struggles with equations requiring multiple variables or precise constants.

On remote sensing datasets involving more than ten input variables, DragonSR demonstrates a clear advantage: EVI2 (9/10), SAVI (10/10), and AWEI_sh (10/10*) are recovered at $\epsilon = 0$, and strong performance persists up to 10% noise on several indices. PySR recovers NDVI (3/10*),

AWEI_sh (8/10), and NIRv (1/10*) at $\epsilon = 0$ but achieves no recovery at $\epsilon > 0$ on any remote sensing target. However, certain targets remain challenging for both methods, particularly those with complex transcendental structure (Schechter, Planck) or requiring precise numerical constants (Newton, WI2015). **TODO: Note that PySR was run with 2000 iterations versus DragonSR’s 10 000 (Table 6), which may understate PySR’s capabilities on harder targets.**

5 Conclusion

This work presented DragonSR, a symbolic regression method that searches over directed acyclic graphs with multi-channel outputs. The method combines optional target denoising, feature augmentation and selection, a complexity-structured search driven by Mutant-UCB [32], and an aggregation mechanism that combines the symbolic channels produced by each candidate DAG. Depending on the selected configuration, candidate graphs may also contain learnable parameters, which are optimized during their evaluation. The final symbolic expression is extracted directly from the candidate DAG achieving the lowest search loss over the explored complexity levels.

Within the considered benchmark and experimental settings, DragonSR recovered expressions across the Nguyen, physics-inspired, and remote-sensing categories. It obtained higher recovery rates than PySR on most noisy benchmarks and on several remote-sensing expressions involving a relatively large number of input variables. PySR nevertheless achieved strong recovery on several noiseless targets, while both methods encountered difficulties on expressions involving complex transcendental structures or precise numerical constants. These results indicate that DragonSR is a viable symbolic regression approach for the considered problems, but they do not establish that its DAG representation is systematically preferable to tree-based representations.

The computational budgets of the two methods are not directly expressed in the same units. DragonSR was assigned a global budget of (10,000) candidate evaluations, including the initial population and the candidates evaluated across successive complexity levels. PySR was run for (2,000) iterations with five populations of 120 individuals. A PySR iteration evolves its populations through multiple mutation and evaluation cycles and can also include numerical constant optimization. Consequently, the lower number of reported PySR iterations does not imply a proportionally smaller computational budget, while multiplying the number of iterations by the number of populations does not provide an exact equivalence with the DragonSR evaluation count. The comparison should therefore be understood as using fixed method-specific search budgets rather than strictly matched numbers of candidate evaluations or wall-clock times.

Several limitations remain. The reported recovery metric considers only the final candidate selected according to each method’s optimization objective and does not compare their complete accuracy–complexity trade-offs. Moreover, the present experiments do not isolate the respective effects of denoising, feature augmentation, complexity-level scheduling, multi-channel aggregation, learnable constants, and the Mutant-UCB search strategy. The results therefore characterize the complete DragonSR pipeline rather than demonstrate the individual benefit of each component.

Future work should include controlled ablation studies and comparisons normalized by wall-clock time or by the number of expression evaluations. It would also be useful to investigate complexity-aware candidate selection, alternative channel-aggregation mechanisms, and more efficient optimization of learnable parameters. Finally, evaluation on additional symbolic regression benchmarks and real-world datasets would help determine more precisely the settings in which DragonSR provides a consistent practical benefit.

A Benchmarks Equations

A.1 Analytical expressions definition

Table 3: Analytical expressions used in the benchmark suite.

Category	Name	Expression
Nguyen	N4	$x^6 + x^5 + x^4 + x^3 + x^2 + x$
	N5	$\sin(x^2) \cos(x) - 1$
	N6	$\sin(x) + \sin(x + x^2)$
	N7	$\log(x + 1) + \log(x^2 + 1)$
	N8	$\sqrt{x}$
	N9	$\sin(x) + \sin(y^2)$
	N10	$2 \sin(x) \cos(y)$
	N11	x^y
	N12	$x^4 - x^3 + \frac{y^2}{2} - y$
Physics	Hubble	$v = 70d$
	Newton	$F = \frac{Gm_1m_2}{r^2}$
	Rydberg	$\Delta E = R \left(\frac{1}{n_1^2} - \frac{1}{n_2^2} \right)$
	Ideal Gas	$V = \frac{nRT}{P}$
	Kepler 3rd	$T = 2\pi \sqrt{\frac{a^3}{GM_\odot}}$
	Bode's Law	$a_n = 0.4 + 0.3 \cdot 2^n$
	Schechter	$\phi(L) = \phi^* \left(\frac{L}{L^*} \right)^\alpha e^{-L/L^*}$
	Leavitt	$M = -2.81 \log_{10}(P) - 1.43$
	Planck	$B(\nu, T) = \frac{2h\nu^3}{c^2(e^{h\nu/kT} - 1)}$
Remote Sensing	NDVI	$\frac{B_8 - B_4}{B_8 + B_4}$
	WI2015	$1.7204 + 171(B_2 + B_3 + B_4) - 3B_2B_3$
	AWEI_sh	$B_2 + 2.5B_3 - 1.5(B_{11} + B_{12}) - 0.25B_8$
	BAI	$\frac{1}{(0.1 - B_4)^2 + (0.06 - B_8)^2}$
	BSI	$\frac{B_{11} + B_4 - B_8 - B_2}{B_{11} + B_4 + B_8 + B_2}$
	EVI2	$2.5 \frac{B_8 - B_4}{B_8 + 2.4B_4 + 1}$
	VARI	$\frac{B_3 + B_4 - B_2}{B_8 - B_4}$
	SAVI	$1.5 \frac{B_8 + B_4 + 0.5}{B_8 - B_4}$
	NIRv	$B_8 \frac{B_8 - B_4}{B_8 + B_4}$

A.2 Benchmarks sampling distributions

Table 4: Sampling distributions used to generate the benchmark datasets.

Category	Name	Input variable distributions
Nguyen	N4	$x \sim \mathcal{U}(-1, 1)$
	N5	$x \sim \mathcal{U}(-1, 1)$
	N6	$x \sim \mathcal{U}(-1, 1)$
	N7	$x \sim \mathcal{U}(0, 2)$
	N8	$x \sim \mathcal{U}(0, 4)$
	N9	$x, y \sim \mathcal{U}(-1, 1)$
	N10	$x, y \sim \mathcal{U}(-1, 1)$
	N11	$x, y \sim \mathcal{U}(1, 2)$
	N12	$x, y \sim \mathcal{U}(-1, 1)$
Physics	Hubble	$d \sim \mathcal{U}(1, 1000)$
	Newton	$m_1, m_2 \sim \mathcal{U}(10^{24}, 10^{30}), \quad r \sim \mathcal{U}(10^8, 10^{12})$
	Rydberg	$n_1 \sim \mathcal{U}_{\mathbb{Z}}(1, 4), \quad n_2 = n_1 + \mathcal{U}_{\mathbb{Z}}(1, 4)$
	Ideal Gas	$P \sim \mathcal{U}(10^4, 10^6), \quad n \sim \mathcal{U}(0.1, 10), \quad T \sim \mathcal{U}(200, 1000)$
	Kepler 3rd	$a \sim \mathcal{U}(1.496 \times 10^{10}, 7.48 \times 10^{12})$
	Bode's Law	$n \in \{0, \dots, 8\}^*$
	Schechter	$L \sim \text{Exp}(10^{10}), \quad L \in [10^6, 10^{13}]$
	Leavitt	$P \sim \mathcal{U}(1, 100)$
Remote Sensing	Planck	$\nu \sim \mathcal{U}(10^{11}, 3 \times 10^{14}), \quad T \sim \mathcal{U}(1000, 30000)$
	–	Sentinel-2 bands from satellite observations

*The Bode's law benchmark differs from the other synthetic physical laws as it is based on a finite set of planetary observations rather than randomly sampled variables.

B End-to-end example on Sentinel-2 data

Figure 8 traces the complete DragonSR pipeline on a remote-sensing input, from raw features to the evaluation of a candidate DAG.

Step **(1)** corresponds to the preprocessing stage of Section 3.3. Variable augmentation is applied to the raw Sentinel-2 bands, producing candidate features such as $B_{11} + B_4$ and $B_2 + B_8$ alongside the original bands, latitude and longitude. The XGBoost-based ranking then retains the ten most important features, ordered by increasing importance, which form the input tensor X of the symbolic search.

Step **(2)** shows the evaluation of one candidate DAG on this reduced feature set, following the recursive computation of Section 3.2. Each edge is annotated with the channel representation it carries. The example illustrates the three mechanisms introduced in Sections 3.2 and 3.4: **SelectFeatures** nodes restrict the channels to a learned subset (e.g., $\{0, 4, 6, 7\}$), element-wise primitives such as **Inverse** and **Power** transform every channel simultaneously, and the **add** and **mul** combiners merge parent branches of different channel dimensions after padding (Appendix F). The last node outputs the final multi-channel representation, from which the symbolic expression of each channel can be read directly.

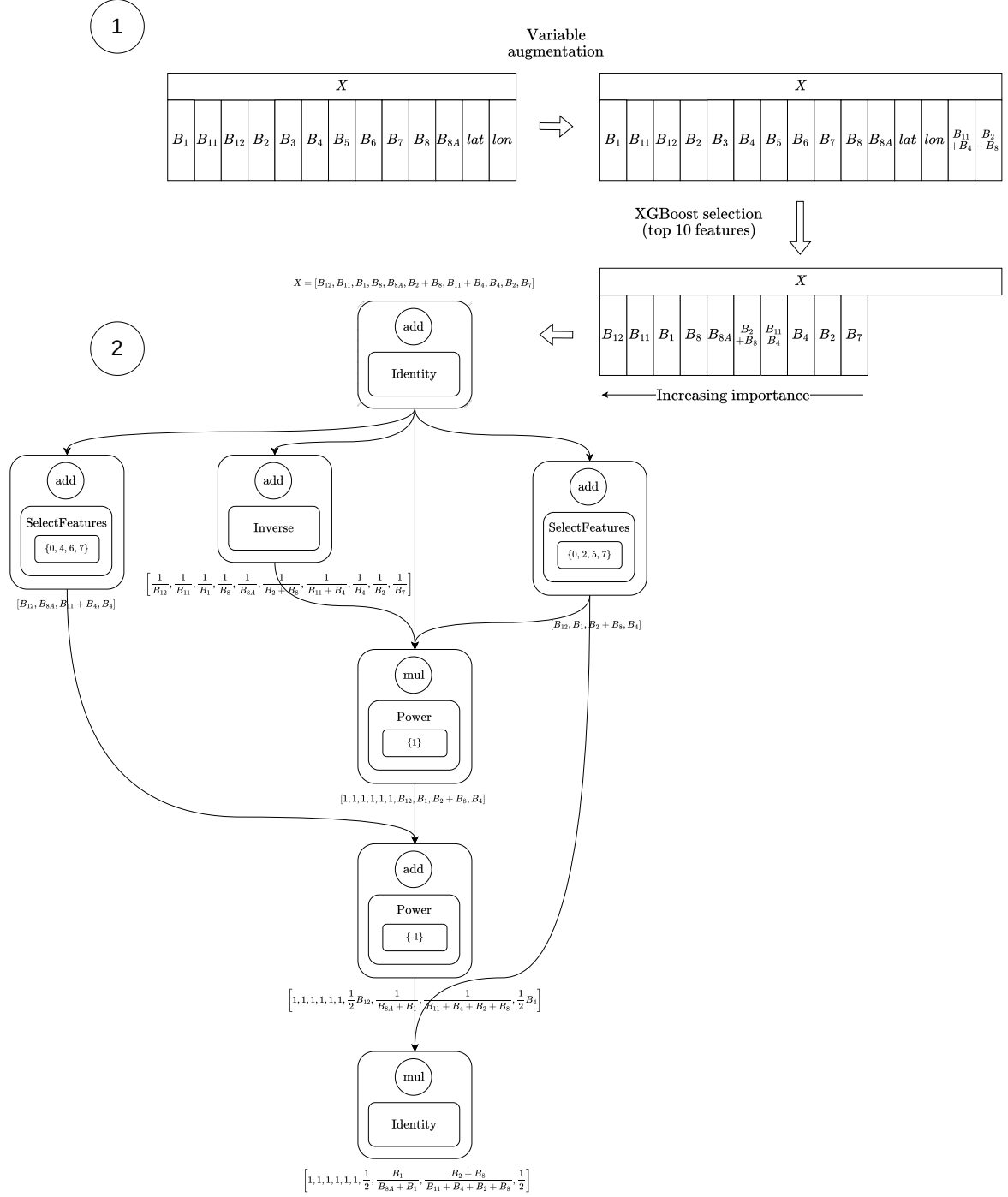


Figure 8: End-to-end example of the DragonSR pipeline on Sentinel-2 bands. **(1)** Preprocessing: variable augmentation followed by XGBoost-based selection of the top-10 features (Section 3.3). **(2)** Evaluation of a candidate DAG on the selected features: edges are annotated with the intermediate channel representations produced at each node, showing the effect of **SelectFeatures**, element-wise primitives, and combiners on the channel structure (Sections 3.2 and 3.4).

C DragonSR search space

Table 5: Search primitives and combiners available in DragonSR. Unary primitives are applied element-wise to the input feature tensor. Binary primitives operate element-wise on pairs of feature tensors, whereas combiners perform tensor-level operations. 

Category	Primitive	Operation
Feature	SelectFeatures	Feature selection
Feature	SplitFeatures	Split input features along the channel dimension
Feature	SumFeatures	Channel-wise summation over input features
Feature	SwitchFeatures	Permutation of selected feature channels
Feature	ChannelBoost	Channel-wise scaling
Unary	Identity	$\mathbf{x}$
Unary	Inverse	$1/\mathbf{x}$
Unary	Negate	$-\mathbf{x}$
Unary	Power	$\mathbf{x}^p$, $p \in \{-3, -2, -1, 1, 2, 3\}$
Unary	Ln	$\ln(\mathbf{x})$
Unary	Exp	$\exp(\mathbf{x})$
Unary	Sin	$\sin(\mathbf{x})$
Unary	Cos	$\cos(\mathbf{x})$
Unary	Sqrt	$\sqrt{\mathbf{x}}$
Binary	Divide	$\mathbf{x}_1 \oslash \mathbf{x}_2$
Binary	Subtract	$\mathbf{x}_1 - \mathbf{x}_2$
Constant	ConstantBrick	Learnable constant
Combiner	add	$\mathbf{X} + \mathbf{Y}$
Combiner	mul	$\mathbf{X} \odot \mathbf{Y}$

D Primitive operator set for variable augmentation

Let $\mathcal{O}_{\text{VA}}$ denote the set of primitive operators used in the variable augmentation procedure. These operators are applied independently to the input features of the dataset $X = (x_1, \dots, x_d) \in \mathbb{R}^{N \times d}$, where each feature $x_i \in \mathbb{R}^N$ is considered as a vector evaluated over the N observations. Each primitive operator defines an element-wise transformation

$$o : \mathbb{R}^N \rightarrow \mathbb{R}^N,$$

and is applied to the input variables to generate additional features for the augmented representation.

The operator set is defined as:

$$\mathcal{O}_{\text{VA}} = \{+, -, \times, \div, \sqrt{\cdot}, \log, |\cdot|, (\cdot)^2, (\cdot)^3, (\cdot)^{-1}\}.$$

For binary operators, the transformation is applied between two input feature vectors, $o(x_i, x_j) : \mathbb{R}^N \times \mathbb{R}^N \rightarrow \mathbb{R}^N$, while unary operators operate on a single feature vector, $o(x_i) : \mathbb{R}^N \rightarrow \mathbb{R}^N$. The resulting features are concatenated with the original input representation to form the augmented input space described in Section 3.3.

E Experimental Setup

Unless otherwise specified, all DragonSR experiments were performed with $N_{\text{iter}} = 10\,000$, $K_{\text{init}} = 30$, a maximum graph complexity of 55, feature sampling enabled, variable augmentation enabled, GPR denoising, parallel execution, and random initialization. All configuration-dependent parameters are summarized below.

E.1 Search Space and Computational Budget

Table 6: Comparison of the search spaces and computational budgets used by PySR and DragonSR. Configuration-dependent parameters are specified in Table 7.

Parameter	PySR	DragonSR
Binary operators	$+, -, \times, \div$	Divide, Subtract
Unary operators	$\log, \exp, \sin, \cos, \sqrt{\cdot}, \cdot $	Configuration-dependent
Power operators	–	$x^k, k \in \{\pm 1, \pm 2, \pm 3\}$
Other primitives	Constant	SelectFeatures, ChannelBoost, Constant
Population size	120	Configuration-dependent
Parallel populations	5	Configuration-dependent
Iterations	2 000	10 000
Maximum complexity	55	55
Iterations per complexity level	–	Configuration-dependent

TODO: Note the budget disparity – PySR runs 2 000 iterations while DragonSR runs 10 000. This may understate PySR’s capabilities on harder targets. A follow-up with equal budgets would strengthen the comparison.

E.2 DragonSR Configurations

Table 7: DragonSR configurations. Parameters not listed remain identical across all experiments.

Config	Op. set	Mode	Loss	Penalty	Parallel pop.	Iter./level / Budget	Const. opt.
A	All	channel	mse	0	4	200 / 10k	No
B	All	channel	corr	0	4	200 / 10k	No
C	All	channel	corr	0.01	8	20 / 10k	No
D	All+Boost	channel	corr	0.01	8	5 / 10k	No
E	All+Boost	channel	mse	0.01	8	5 / 10k	No
F	All+Boost	ols	mse	0.01	8	5 / 10k	No
G	All+Boost	ols	corr	0.01	8	5 / 10k	No
H	All+Boost	channel	corr	0.01	4	5 / 10k	Yes
I	All+Boost	full	corr	0.01	4	5 / 2k	No
J	All	channel	mse	0.01	8	20 / 10k	No

Operator set definitions. All denotes the full set of symbolic primitives, including select, unary operators (identity, inverse, negation), power operators ($x^k, k \in \{\pm 1, \pm 2, \pm 3\}$), logarithmic and exponential functions ($\ln, \exp$), trigonometric functions ($\sin, \cos$), and constant nodes.

All+Boost extends All by additionally including the **ChannelBoost** operator, which constructs augmented channels via learned combinations of input features.

Constant optimization. All configurations except I do not perform any constant optimization. For configuration I values of constant nodes are optimized using a dichotomy-based procedure

during the symbolic search.

E.3 Benchmark Configurations

Table 8: Configuration mapping as a function of dataset and noise level. Each entry refers to Table 7. Entries marked “–” correspond to runs that did not follow the 10k-iteration protocol and are being rerun.

Dataset	$\epsilon = 0$	$\epsilon = 0.01$	$\epsilon = 0.05$	$\epsilon = 0.10$
<i>Nguyen</i>				
Nguyen-4	I	A	B	B
Nguyen-5	J	C	C	D
Nguyen-6	G	D	D	D
Nguyen-7	–	D	D	D
Nguyen-8	I	D	D	D
Nguyen-9	I	D	D	D
Nguyen-10	D	D	D	D
Nguyen-11	D	E	E	E
Nguyen-12	D	F	F	F
<i>Physics</i>				
Hubble	I	H	H	H
Newton	–	H	H	H
Rydberg	G	D	D	D
Ideal Gas	I	D	D	D
Kepler 3rd	–	D	D	D
Bode’s Law	G	H	H	H
Schechter	–	H	H	H
Leavitt	I	H	H	H
Planck’s Law	–	H	H	H
<i>Remote Sensing</i>				
NDVI	D	D	C	C
WI2015	D	H	H	H
AWEI_sh	I	G	G	G
BAI	D	D	D	D
BSI	D	C	D	D
EVI2	I	H	H	H
VARI	D	H	H	H
SAVI	I	H	H	H
NIRv	D	D	D	D

TODO: add the method for all runs

F Combiners padding

Let

$$d_{\max} = \max_j \dim(S_{i_j}),$$

where $\dim(\cdot)$ denotes the tensor length. Each tensor S_{i_j} is then left-padded with the neutral element associated with the combiner until its dimension equals $d_{\max}$:

$$S_{i_j} \mapsto (\underbrace{e, \dots, e}_{d_{\max} - \dim(S_{i_j})}, S_{i_j}), \quad \text{where } e = \begin{cases} 0, & \diamond = +, \\ 1, & \diamond = \times. \end{cases}$$

Consequently, the combination is always performed on tensors of identical dimensions.

G Feature Selection Criterion

The variable augmentation procedure evaluates each candidate feature according to its squared Pearson correlation with the target variable. Given a candidate feature $c \in \mathbb{R}^N$ and a target vector $y \in \mathbb{R}^N$, the squared Pearson correlation coefficient is defined as:

$$R^2(c, y) = \left(\frac{\sum_{i=1}^p (c_i - \bar{c})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^p (c_i - \bar{c})^2 \sum_{i=1}^p (y_i - \bar{y})^2}} \right)^2,$$

where $\bar{c}$ and $\bar{y}$ denote the empirical means of the candidate feature and the target variable, respectively. This criterion measures the linear dependence between a generated feature and the target, with higher values indicating stronger correlation.

H Exploratory methods not retained

This appendix briefly documents two exploratory directions that were not retained in the final experimental pipeline. These attempts were conducted during method development, but were not evaluated through the same systematic protocol as the experiments reported in the main leaderboard. Consequently, the observations below should not be interpreted as controlled ablation results or as evidence that these approaches are ineffective in general.

H.1 MC-Dropout uncertainty weighting

In addition to the Gaussian-process regression (GPR) preprocessing described in the main experiments, we explored a surrogate-assisted strategy based on Monte Carlo dropout. Unlike the GPR denoiser, this approach did not replace the observed targets with smoothed predictions. Instead, it used the predictive uncertainty of a neural surrogate to assign a confidence weight to each observation.

A multilayer perceptron was trained to approximate the standardized mapping from the input variables to the target. The network consisted of two hidden layers with ReLU activations and dropout applied after each hidden layer. After training, dropout remained active during inference and T_{MC} stochastic forward passes were performed. For each observation i , the empirical predictive variance was computed as

$$v_i = \frac{1}{T_{\text{MC}}} \sum_{t=1}^{T_{\text{MC}}} \left(\hat{y}_i^{(t)} - \bar{y}_i \right)^2, \quad (3)$$

where $\hat{y}_i^{(t)}$ denotes the prediction obtained during stochastic forward pass t . This variance was converted into a confidence score,

$$c_i = \frac{1}{1 + v_i/\bar{v}}, \quad \bar{v} = \frac{1}{n} \sum_{j=1}^n v_j, \quad (4)$$

and normalized to obtain sample weights with unit mean:

$$w_i = \frac{nc_i}{\sum_{j=1}^n c_j}. \quad (5)$$

Observations associated with high surrogate uncertainty therefore received a smaller weight during the symbolic-regression search. The motivation was to reduce the influence of locally noisy or difficult samples without explicitly smoothing the target.

In preliminary experiments, this weighting strategy did not produce consistently more accurate or simpler expressions than the self-validating GPR preprocessing. It also introduced an additional neural-network training stage and several hyperparameters, including the network width, dropout probability, number of training epochs, and number of stochastic forward passes. Since these exploratory tests were not conducted over the complete benchmark and run protocol, no quantitative comparison is reported. The GPR approach was retained because it provided a simpler and more directly interpretable preprocessing mechanism. The MC-Dropout implementation remains available in the public code repository referenced in Section I.

H.2 Exploratory application to IASI wet-bulb temperature retrieval

We also explored the application of DragonSR to the retrieval of wet-bulb temperature (WBT) from measurements produced by the Infrared Atmospheric Sounding Interferometer (IASI). The dataset combined IASI spectral radiances with colocated meteorological observations. The prediction target was the WBT associated with the ground measurements, while the candidate input variables consisted of spectral channels, with latitude and longitude included in some experiments.

The reference approach developed for this dataset was a multilayer perceptron trained either on a physically motivated selection of 87 spectral channels or on the complete IASI spectrum. The reported validation RMSE was approximately 1.60°C with the selected channels and 1.55°C when using all channels.

Several DragonSR configurations were explored on subsets of the data and with different input representations. These experiments produced both large expressions combining many spectral channels and more compact candidate formulas involving only a small number of channels. However, the highly dimensional and strongly correlated spectral input made the search particularly difficult. The most accurate expressions were generally too large to provide a useful interpretable alternative to the neural network, whereas the compact expressions did not reach a satisfactory predictive accuracy.

These experiments remained exploratory: the candidate expressions were generated and inspected under different preprocessing choices, sample subsets, and search configurations, and were therefore not evaluated under a sufficiently uniform protocol for a formal comparison with the neural-network baseline. We consequently exclude this application from the main experimental

results. Nevertheless, it illustrates an important limitation of the present approach: on high-dimensional inverse problems in which the target depends on many correlated measurements, obtaining an expression that is simultaneously compact, physically interpretable, and competitive with a flexible neural predictor remains challenging.

I Data availability

The complete run-level outputs used to compute the results reported in the leaderboard are publicly available on Zenodo at <https://doi.org/10.5281/zenodo.22146752>. The archive contains the DragonSR and PySR outputs for every evaluated benchmark and noise level, organized by method, noise condition, benchmark expression, and independent run.

The code used to run the leaderboard experiments is available at <https://github.com/JulieKeisler/DRAGON/tree/dev-elyas/leaderboard>. This unified experimental pipeline supports running DragonSR, PySR, or both methods successively, using the corresponding configuration files. It also handles dataset generation and preprocessing, independent runs, noise injection, result collection, and leaderboard generation.

References

- [1] John R. Koza. Genetic programming as a means for programming computers by natural selection. *Statistics and Computing*, 4(2):87–112, June 1994. doi: 10.1007/BF00175355.
- [2] Michael Schmidt and Hod Lipson. Distilling free-form natural laws from experimental data. *Science*, 324(5923):81–85, April 2009. doi: 10.1126/science.1165893.
- [3] Miles Cranmer. Interpretable machine learning for science with pysr and symbolicregression.jl. *arXiv preprint arXiv:2305.01582*, 2023.
- [4] Riccardo Poli, William B. Langdon, and Nicholas Freitag McPhee. *A Field Guide to Genetic Programming*. Lulu.com, 2008. ISBN 978-1-4092-0073-4. URL <http://www.gp-field-guide.org.uk/>.
- [5] Tony Worm and Kenneth Chiu. Prioritized grammar enumeration: Symbolic regression by dynamic programming. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2013.
- [6] Silviu-Marian Udrescu and Max Tegmark. Ai feynman: a physics-inspired method for symbolic regression. *arXiv preprint arXiv:1905.11481*, 2019. URL <https://arxiv.org/abs/1905.11481>.
- [7] Brenden K. Petersen, Mikel Landajuela, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://openreview.net/forum?id=m5Qsh0kBQG>.
- [8] Mojtaba Valipour, Bowen You, Maysum Panju, and Ali Ghodsi. Symbolicgpt: A generative transformer model for symbolic regression. *arXiv preprint arXiv:2106.14131*, 2021.
- [9] Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K. Reddy. Llm-sr: Scientific equation discovery via programming with large language models. In *International Conference on Learning Representations (ICLR)*, 2025.
- [10] Yanjie Li, Weijun Li, Lina Yu, Min Wu, Jingyi Liu, Wenqiang Li, Shu Wei, and Yusong Deng. Mllm-sr: Conversational symbolic regression base multi-modal large language models. *arXiv preprint arXiv:2406.05410*, 2024.
- [11] Julie Keisler, El-Ghazali Talbi, Sandra Claudel, and Gilles Cabriel. An algorithmic framework for the optimization of deep neural networks architectures and hyperparameters. *Journal of Machine Learning Research*, 25(201):1–33, 2024. URL <https://www.jmlr.org/papers/v25/23-0166.html>.
- [12] Silviu-Marian Udrescu, Andrew Tan, Jiahai Feng, Orisvaldo Neto, Tailin Wu, and Max Tegmark. AI Feynman 2.0: Pareto-optimal symbolic regression exploiting graph modularity. In *Advances in Neural Information Processing Systems*, volume 33, pages 4860–4871, 2020. arXiv:2006.10782.
- [13] Donald Gerwin. Information processing, data inferences, and scientific generalization. *Behavioral Science*, 19(5):314–325, 1974.
- [14] Pat Langley. Bacon: A production system that discovers empirical laws. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 1977.

- [15] Pat Langley. Rediscovering physics with bacon.3. In *Proceedings of the 6th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 505–507. Morgan Kaufmann Publishers Inc., 1979.
- [16] Pat Langley, Gary L. Bradshaw, and Herbert A. Simon. Bacon.5: The discovery of conservation laws. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 1981.
- [17] Pat Langley and Jan M. Zytkow. Data-driven approaches to empirical discovery. *Artificial Intelligence*, 40(1):283–312, 1989.
- [18] Josh Bongard and Hod Lipson. Automated reverse engineering of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 104(24):9943–9948, June 2007. doi: 10.1073/pnas.0609476104.
- [19] Michael Schmidt and Hod Lipson. Symbolic regression of implicit equations. In *Genetic Programming Theory and Practice VII*, pages 73–85. Springer, 2010.
- [20] Stefan Wagner and Michael Affenzeller. Heuristiclab: A generic and extensible optimization environment. In Bernardete Ribeiro, Rudolf F. Albrecht, Andrej Dobnikar, David W. Pearson, and Nigel C. Steele, editors, *Adaptive and Natural Computing Algorithms*, pages 538–541. Springer, 2005.
- [21] Trevor Stephens. gplearn: Genetic programming in python, with a scikit-learn inspired api. <https://github.com/trevorstephens/gplearn>, 2016. GitHub repository.
- [22] Bogdan Burlacu, Gabriel Kronberger, and Michael Kommenda. Operon C++: An efficient genetic programming framework for symbolic regression. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference Companion, GECCO ’20*, pages 1562–1570. ACM, 2020. doi: 10.1145/3377929.3398099.
- [23] Julian F. Miller and Peter Thomson. Cartesian genetic programming. In *Genetic Programming: European Conference, EuroGP 2000*, volume 1802 of *Lecture Notes in Computer Science*, pages 121–132. Springer, 2000. doi: 10.1007/978-3-540-46239-2_9.
- [24] Dennis G. Wilson, Julian F. Miller, Sylvain Cussat-Blanc, and Hervé Luga. Positional Cartesian genetic programming. *arXiv preprint arXiv:1810.04119*, 2018.
- [25] Paul Kahlmeyer, Joachim Giesen, Michael Habeck, and Henrik Voigt. Scaling up unbiased search-based symbolic regression. *arXiv preprint arXiv:2506.19626*, 2025. doi: 10.48550/arXiv.2506.19626. URL <https://arxiv.org/abs/2506.19626>.
- [26] Trent McConaghy. FFX: Fast, scalable, deterministic symbolic regression technology. *Genetic Programming Theory and Practice IX*, pages 235–260, 2011.
- [27] Thomas R. Harvey, Fabian Rühle, Cristoforo S. Fraser-Taliente, and James Halverson. Symbolic regression with multimodal large language models and kolmogorov-arnold networks. *arXiv preprint arXiv:2505.07956*, 2025.
- [28] Nguyen Quang Uy, Nguyen Xuan Hoai, Michael O’Neill, Robert I. McKay, and Edgar Galván-López. Semantically-based crossover in genetic programming: application to real-valued symbolic regression. *Genetic Programming and Evolvable Machines*, 12(2):91–119, 2011.
- [29] T. Nathan Mundhenk, Mikel Landajuela, Ruben Glatt, Claudio P. Santiago, Daniel Faissol, and Brenden K. Petersen. Symbolic regression via neural-guided genetic programming population seeding. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. arXiv:2111.00053.

- [30] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H. Moore. SRBench: A comprehensive benchmark for symbolic regression. *arXiv preprint arXiv:2106.06427*, 2021.
- [31] Frank Hutter, Lars Kotthoff, and Joaquin Vanschoren, editors. *Automated Machine Learning: Methods, Systems, Challenges*. Springer, 2019.
- [32] Margaux Brégère and Julie Keisler. A bandit approach with evolutionary operators for model selection: Application to neural architecture optimization for image classification. *arXiv preprint arXiv:2402.05144*, 2024. doi: 10.48550/arXiv.2402.05144. URL <https://arxiv.org/html/2402.05144v1>.
- [33] Fabrício Olivetti de França and Gabriel Kronberger. Equality graph assisted symbolic regression. *arXiv preprint arXiv:2511.01009*, 2025.
- [34] Nathan Haut, Wolfgang Banzhaf, and Bill Punch. Correlation versus rmse loss functions in symbolic regression tasks. *arXiv preprint arXiv:2205.15990*, 2022. doi: 10.48550/arXiv.2205.15990. URL <https://arxiv.org/abs/2205.15990>.
- [35] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785. URL <https://doi.org/10.1145/2939672.2939785>.
- [36] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [37] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006. ISBN 026218253X.